

第四章 存储管理

by Jiacheng Xu

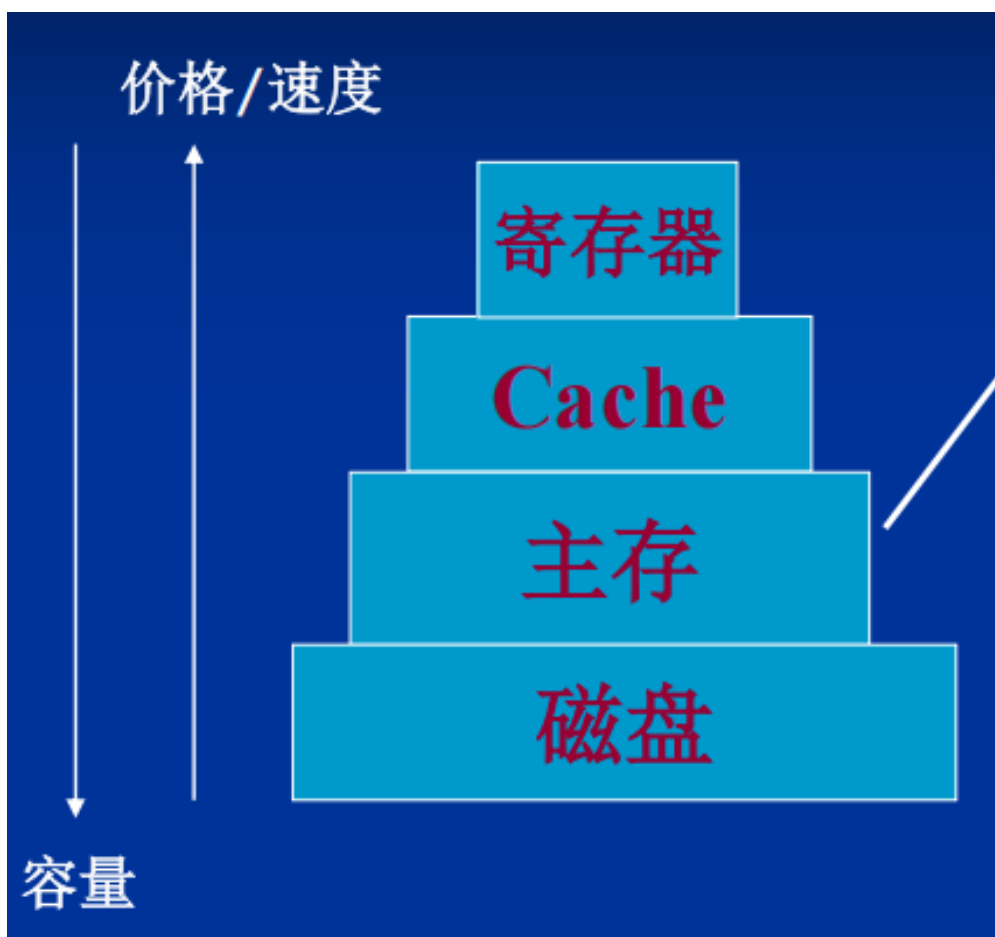
- 第四章 存储管理
 - 4.0 存储体系概述
 - 4.1 程序的装入和链接
 - 4.1.0 程序的编程、编译、链接和装入
 - 4.1.1 程序的装入（进入物理内存的部分）
 - (1) 绝对装入方式
 - (2) 可重定位装入方式
 - (3) 应用虚拟技术实现存储管理（2的延伸）
 - 1. 逻辑地址和物理地址
 - 2. 名空间、逻辑地址空间、物理地址空间之间的关联
 - 3. 地址重定位（地址映射）
 - (4) 动态运行时装入方式
 - 4.1.2 程序的链接
 - 4.2 连续分配方式（分区）
 - 4.2.0 连续分配与离散分配比较
 - 4.2.1 单一连续分配存储管理方案
 - (1) 应用背景
 - (2) 基本思想
 - (3) 实现
 - (4) 特点
 - 4.2.2 固定分区分配存储管理方案
 - (1) 应用背景
 - (2) 基本思想
 - (3) 实现
 - (4) 特点
 - 4.2.3 动态分区分配存储管理方案
 - (1) 应用背景
 - (2) 基本思想
 - (3) 实现
 - 1. 数据结构
 - 2. 算法
 - (a) 分区分配算法——首次适应算法

- (b) 分区分配算法——循环首次适应算法
 - (c) 分区分配算法——最佳适应算法
 - (d) 分区分配算法——最坏适应算法
- (4) 特点
- 4.2.4 可重定位分区分配存储管理方案
 - (1) 应用背景
 - (2) 基本思想
- 4.3 基本分页存储管理方式
 - 4.3.0 分区式分配存在的问题
 - 4.3.1 分页存储的基本设计思想
 - (1) 引入页面与页框
 - (2) 引入页表
 - (3) 页面尺寸
 - (4) 地址结构
 - 4.3.2 地址重定位
 - (1) 问题
 - (2) 基本地址变换机构/地址映射机制
 - (3) 具有快表的地址变换机构
 - 1. 快表的概念
 - 2. 地址变换机构
 - 4.3.3 两级和多级页表
 - (1) 页表的存储问题
 - (2) 两级页表
 - (3) 多级页表
 - 4.4 基本分段存储管理方式
 - 4.4.0 分页系统的不足
 - 4.4.1 设计思想
 - (1) 设计思想概述
 - (2) 分段 (Segment)
 - (3) 段表
 - (4) 地址结构
 - 4.4.2 地址重定位
 - (1) 问题
 - (2) 分段地址变换及存储保护机构
 - (3) 地址变换机构/地址映射机制：计算实例
 - (4) 分段和分页区别
 - 4.4.3 信息共享

- (1) 分页信息共享
 - (2) 分段信息共享
- 4.5 段页式存储管理方式
 - 4.5.0 分段系统存在的问题
 - 4.5.1 设计思想
 - (1) 设计思想概述
 - (2) 段表和页表
 - (3) 地址结构
 - 4.5.2 地址变换
- 4.6 虚拟存储器的基本概念（重重重点，小时候抱过出题老头）
 - 4.6.0 存储管理的两个基本问题
 - 4.6.1 虚拟存储器的理论基础——局部性原理
 - 4.6.2 虚拟存储器概念
 - (1) 基本原理
 - (2) 虚拟存储器的定义
 - (3) 虚拟存储器的特征
 - (4) 虚拟存储器的实现
- 4.7 请求分页存储管理方式
 - 4.7.1 请求分页中的硬件支持
 - (1) 页表机制
 - (2) 缺页中断机制
 - (3) 地址重定位
 - 4.7.2 内存分配策略和算法
 - (1) 最小物理块个数
 - (2) 工作集
 - (3) 物理块分配策略
 - 1. 固定分配局部置换
 - 2. 可变分配全局置换
 - 3. 可变分配局部置换
 - (4) 物理块分配算法
 - 4.7.3 页面策略
 - (1) 何时调入
 - (2) 何处调入
 - 4.7.4 页面置换算法
 - (1) 最佳置换算法
 - (2) 先进先出置换算法（FIFO）
 - (3) 最近最久未使用置换算法

- (4) 置换算法性能比较
- 4.8 请求分段存储管理方式
 - 4.8.1 请求分段中的硬件支持
 - (1) 段表机制
 - (2) 缺段中断机制
 - (3) 地址重定位
 - 4.8.2 分段共享与保护
 - (1) 数据结构
 - (2) 算法：共享分段的分配与回收

4.0 存储体系概述



主存中包含的内容如下

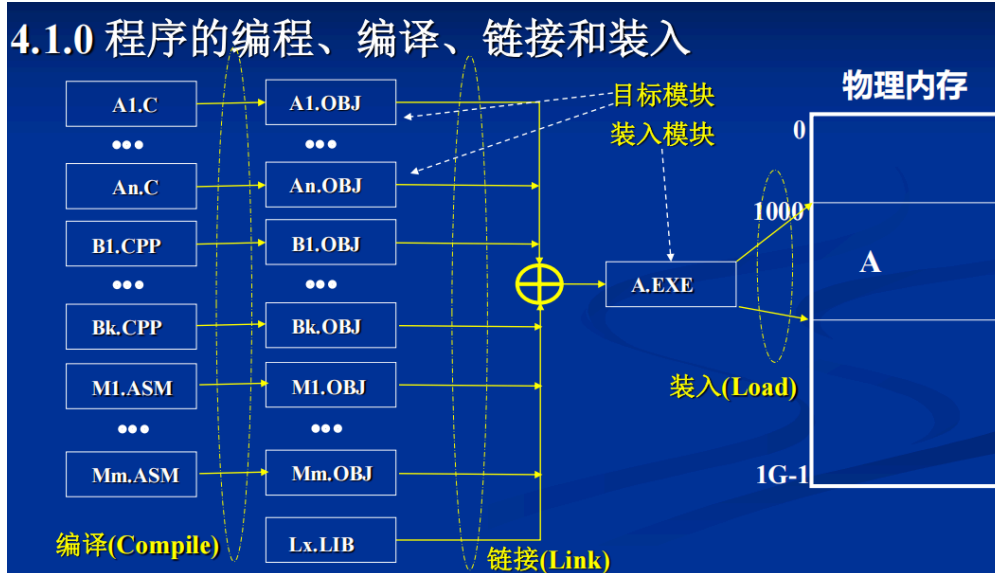
- **内存空间**：存储单元（字节或字）组成的一维连续的地址空间
- **存储器**：存放当前正在运行程序的代码及数据
- 性能瓶颈：关键、紧张
- 帕金森定律：内存多大、程序多长

4.1 程序的装入和链接

4.1.0 程序的编程、编译、链接和装入

过程（理解）：

C/CPP/Compile → 编译(Compile) → OBJ/LIB (目标模块) → 链接(Link) → EXE (装入模块) → 装入 (Load) → 物理内存



4.1.1 程序的装入（进入物理内存的部分）

(1) 绝对装入方式

- **定义：** 编程时，直接确定内存位置
- **特点：** 不灵活，不支持多道程序设计技术

(2) 可重定位装入方式

- **定义：**
 - 采用 **虚拟技术**——编程时使用 **逻辑地址**。运行时使用 **物理地址**
 - 装入时，进行 **地址重定位**
- **特点：** 灵活，支持多道程序设计技术

(3) 应用虚拟技术实现存储管理（2的延伸）

1. 逻辑地址和物理地址

1. 逻辑地址（相对地址、虚地址）

- **定义：**
 - 用户程序经过 **汇编或编译后** 形成目标代码；
 - 目标代码采用 **相对地址** 形式
- **特点**

- 首地址为0
- 其余指令中地址以相对于首地址的偏移为地址
- 无法使用逻辑地

2. 物理地址（绝对地址、实地址）

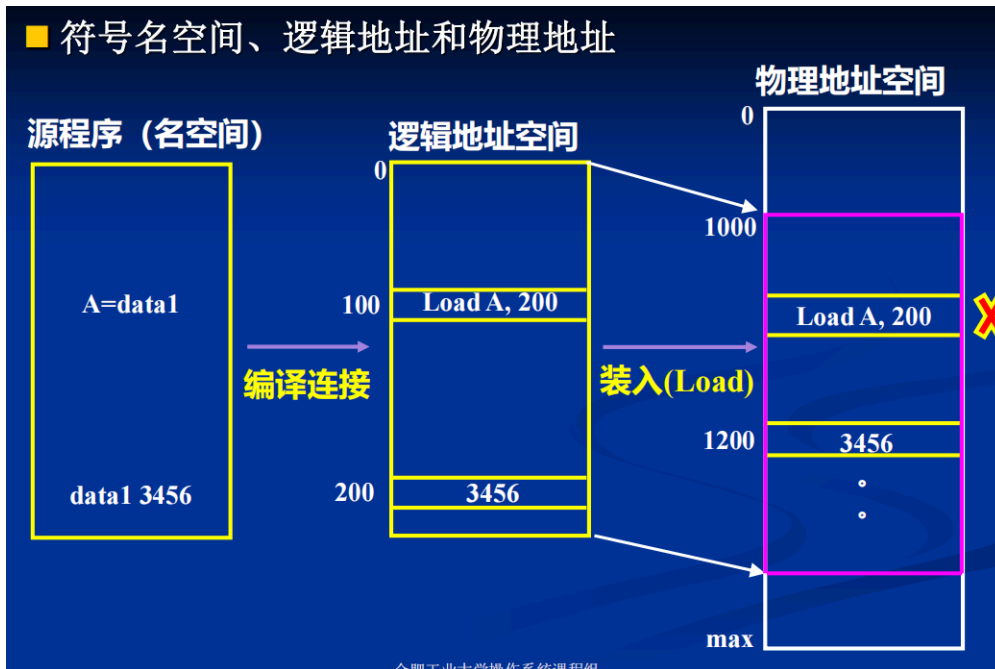
• 定义：

- 内存中存储单元的地址，可直接寻址

2. 名空间、逻辑地址空间、物理地址空间之间的关联

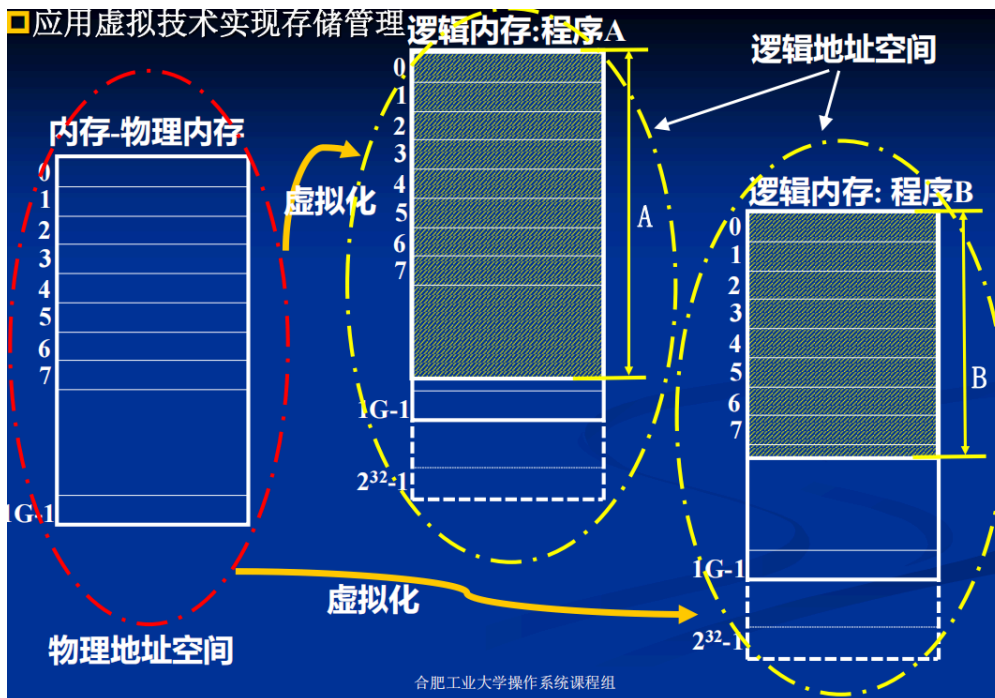
第一张图：三个空间之间的联系

- 这张图中，逻辑地址直接装入物理地址是错误的，见3的地址重定位

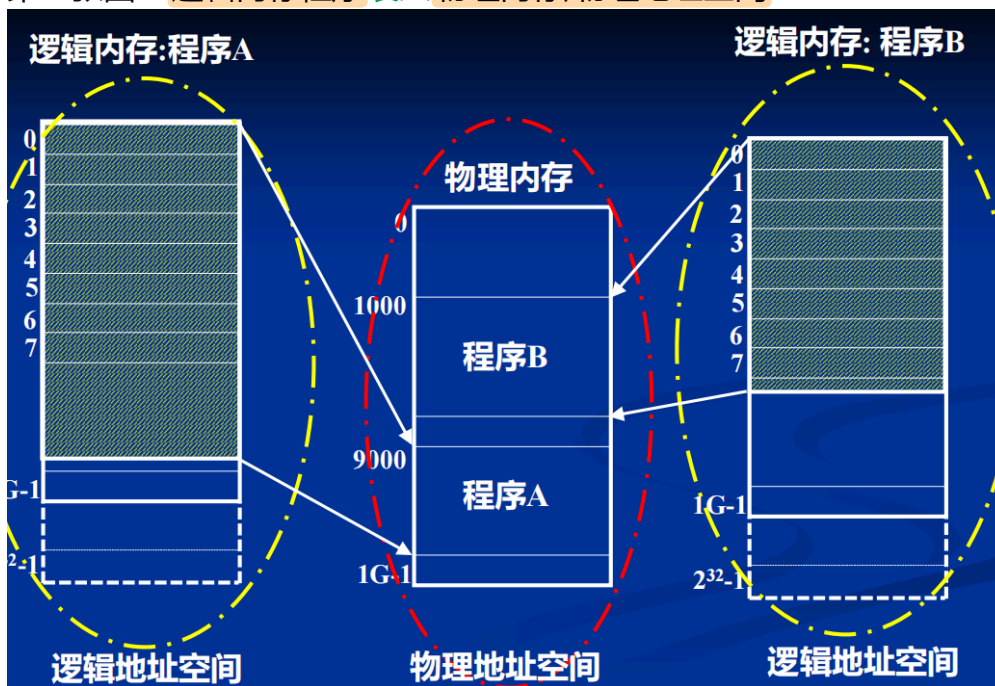


第二张图：物理内存/物理地址空间虚拟化为逻辑内存/逻辑地址空间

应用虚拟技术实现存储管理



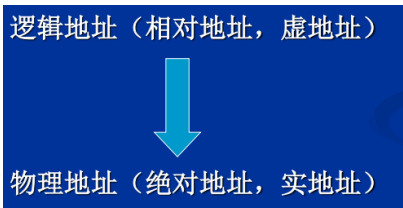
第三张图: 逻辑内存程序装入物理内存/物理地址空间



3. 地址重定位 (地址映射)

定义:

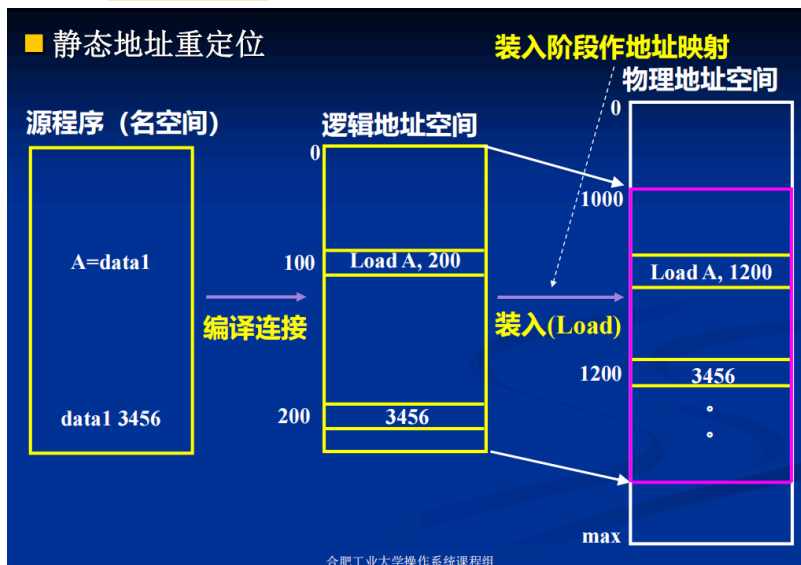
- 为保证CPU执行指令时 **正常访问存储单元**
- 将程序中的逻辑地址 → 机器可直接寻址的物理地址
- 该过程叫 **地址重定位/地址映射**



分类：

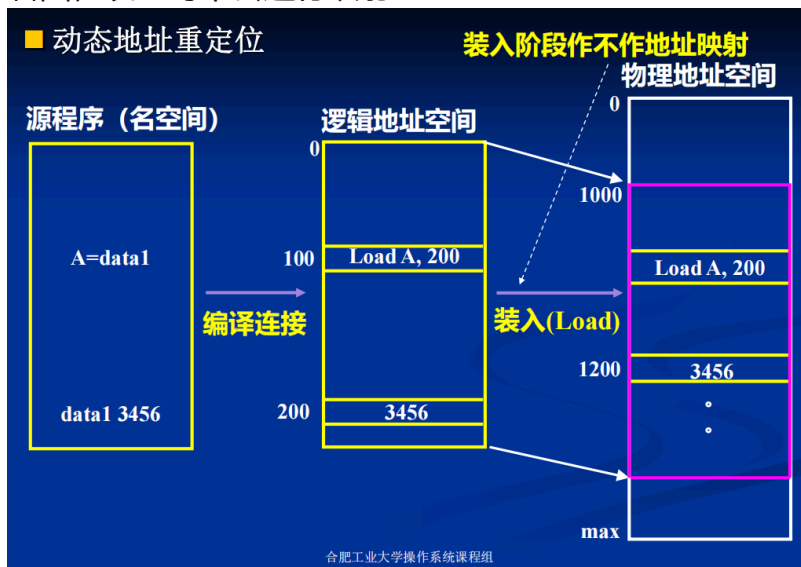
静态地址重定位

- 当用户程序被装入内存时，一次性实现逻辑地址到物理地址的转换，以后不再转换
- 一般在装入内存时由软件完成



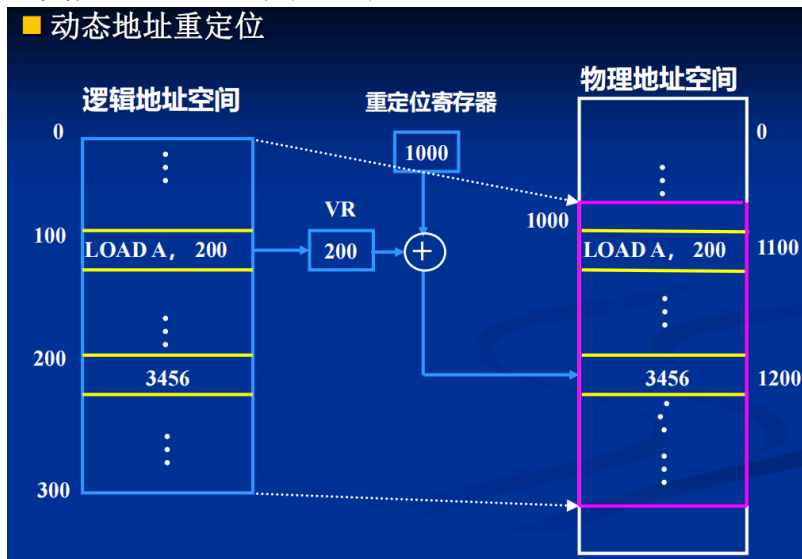
动态地址重定位

- 在程序运行过程中要访问数据时再进行地址变换
- 即在逐条指令执行时完成地址映射。
- 为了提高效率，此工作由硬件地址映射机制来完成。硬件支持，软硬件结合完成
- 硬件上需要一对寄存器的支持
- 看图，装入时不会进行映射



- 看图，需要寄存器实现转换

■ 动态地址重定位



(4) 动态运行时装入方式

- **定义**:
 - 编程时使用**逻辑地址**，运行时使用**物理地址**；
 - **运行时进行地址重定位**（**注意是运行时**）
- **特点**：灵活，支持多道程序设计技术；支持程序“搬家”

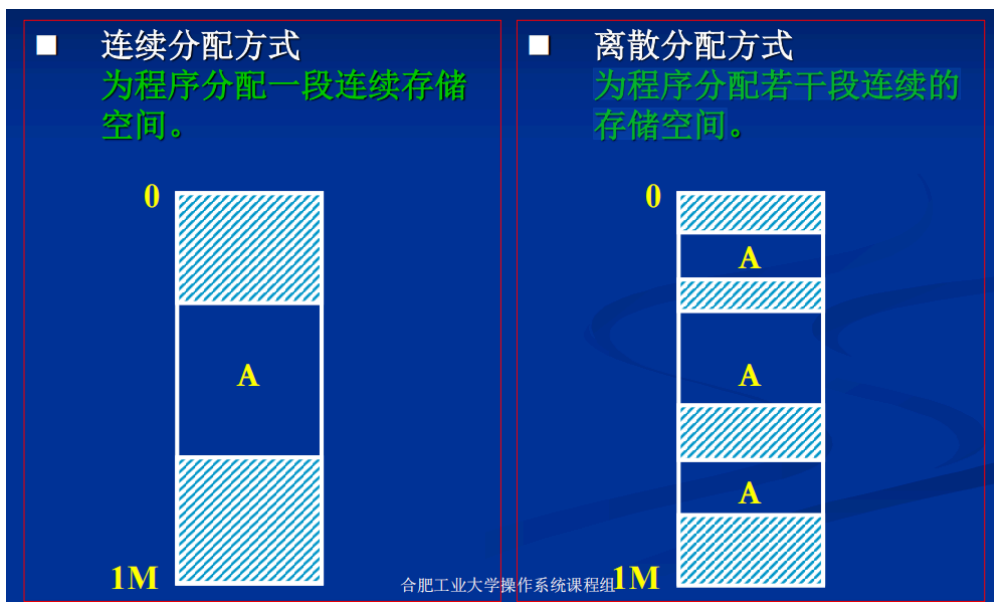
4.1.2 程序的链接

- **静态链接**：**编程**时，链接所有模块；
- **装入时动态链接**：**装入**时，链接所有模块；
- **运行时动态链接**：**运行**时，根据需要链接模块

4.2 连续分配方式（分区）

4.2.0 连续分配与离散分配比较

- **连续分配**：为程序分配**一段连续**存储空间
- **离散分配**：为程序分配**若干段连续**的存储空间



4.2.1 单一连续分配存储管理方案

(1) 应用背景

- 单用户系统
- 在一段时间内，只有一个进程在内存

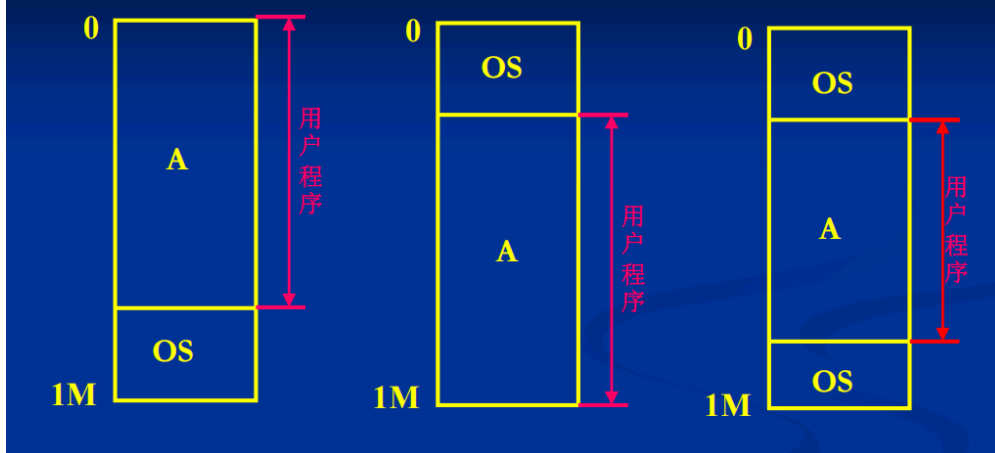
(2) 基本思想

内存分为两个区域：

- 一个给操作系统
- 一个给用户

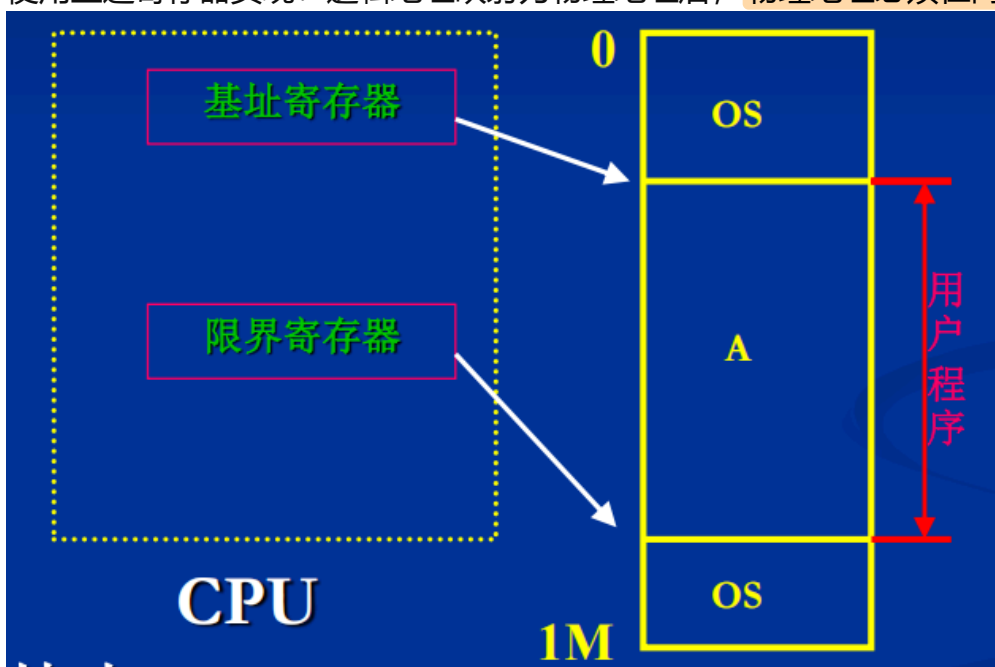


三种典型内存布局：



(3) 实现

- 设置基址寄存器、限界寄存器
- 使用上述寄存器实现：逻辑地址映射为物理地址后，物理地址必须在两者之间



(4) 特点

简单，适用于单任务OS

4.2.2 固定分区分配存储管理方案

(1) 应用背景

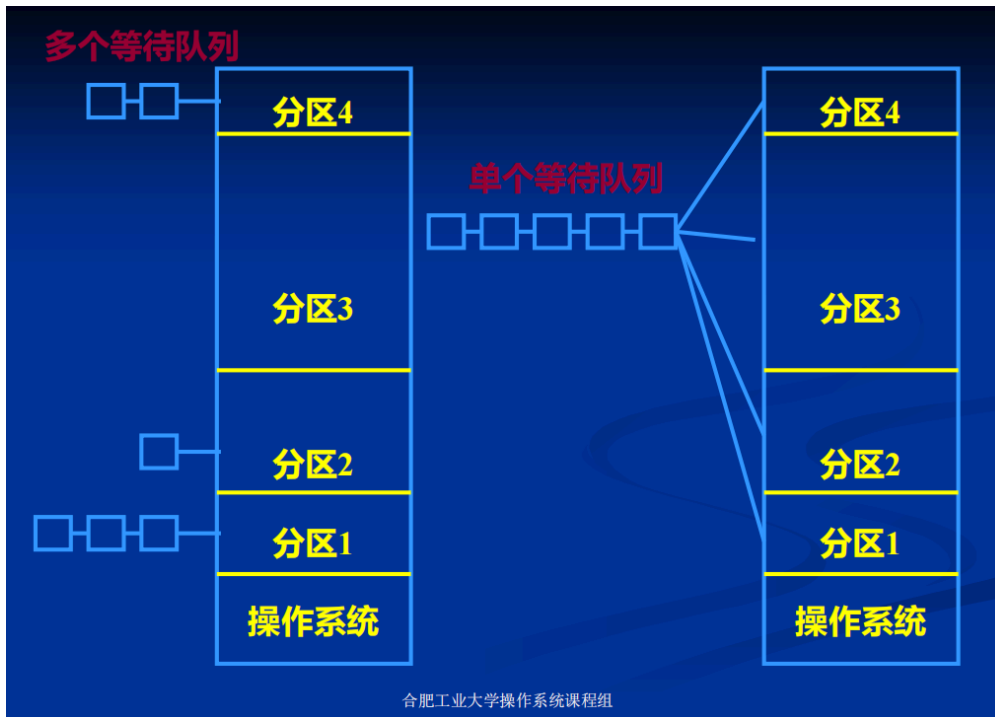
多道程序设计系统

(2) 基本思想

1. **初始化**：用户区划分为若干固定大小的内存区域（分区）

2. 分区特点：

- 各分区可放一个程序运行，程序运行结束后可调入另一个执行
- 划分多个分区，即允许许多个程序同时在内存运行
- 分区大小和位置可不同



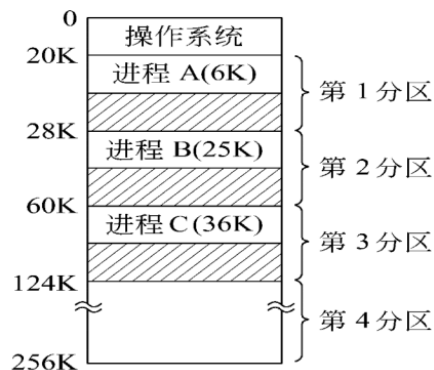
(3) 实现

1. 数据结构：分区说明表

- 看起始地址+分区长度=分配的分区的

区号	分区长度	起始地址	状态
1	8K	20K	已分配
2	32K	28K	已分配
3	64K	60K	已分配
4	132K	124K	未分配

(a) 分区说明表



(b) 内存状态

2. 算法：分区分配算法、分区回收算法

(4) 特点

1. 最早的支持多道程序设计的存储管理方案
2. 简单，现在仍在嵌入式系统中使用
3. "内零头"严重
 - 内零头：分配给进程，而进程未用到的内存部分
 - 外零头：未分配给进程，但因为太小而无进程能用

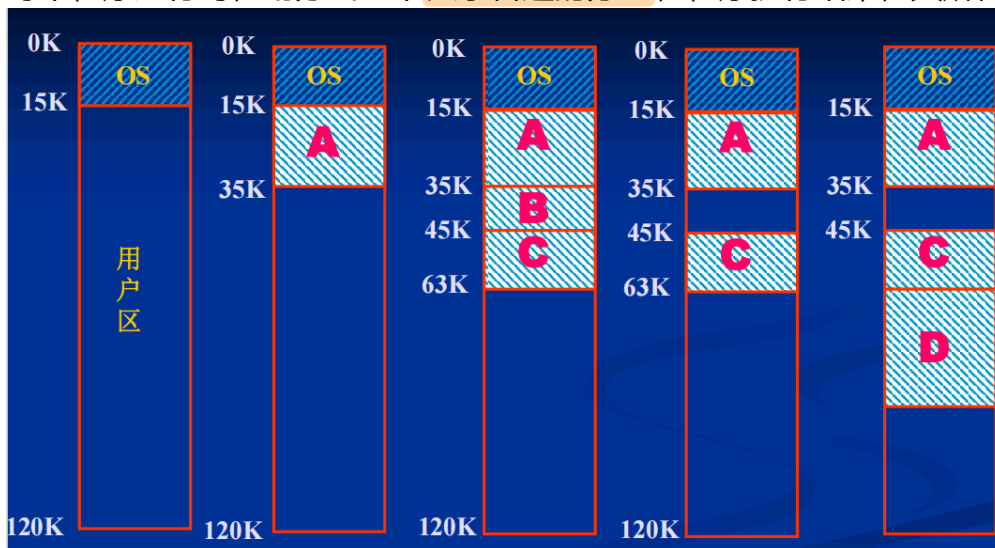
4.2.3 动态分区分配存储管理方案

(1) 应用背景

- 多道程序设计系统

(2) 基本思想

- **总结**：每个程序用一块，动态增减
- OS初始化阶段，将用户区划分为一个大的空白分区；
- 每个程序运行时，划分出一个大小合适的分区；程序执行结束，其所在的分区撤消；

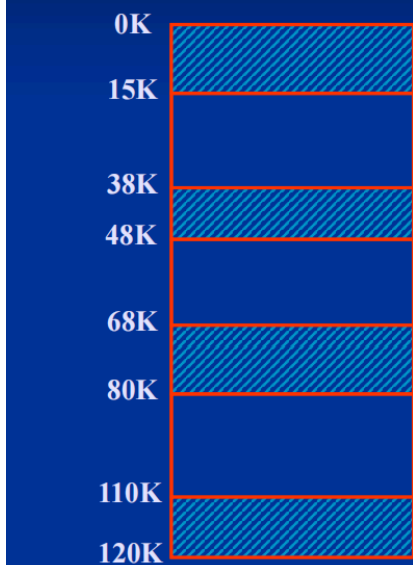


(3) 实现

1. 数据结构

- **空闲分区表**：指明分区位置、长度和分配情况

(1) 数据结构：(a) 空闲分区表



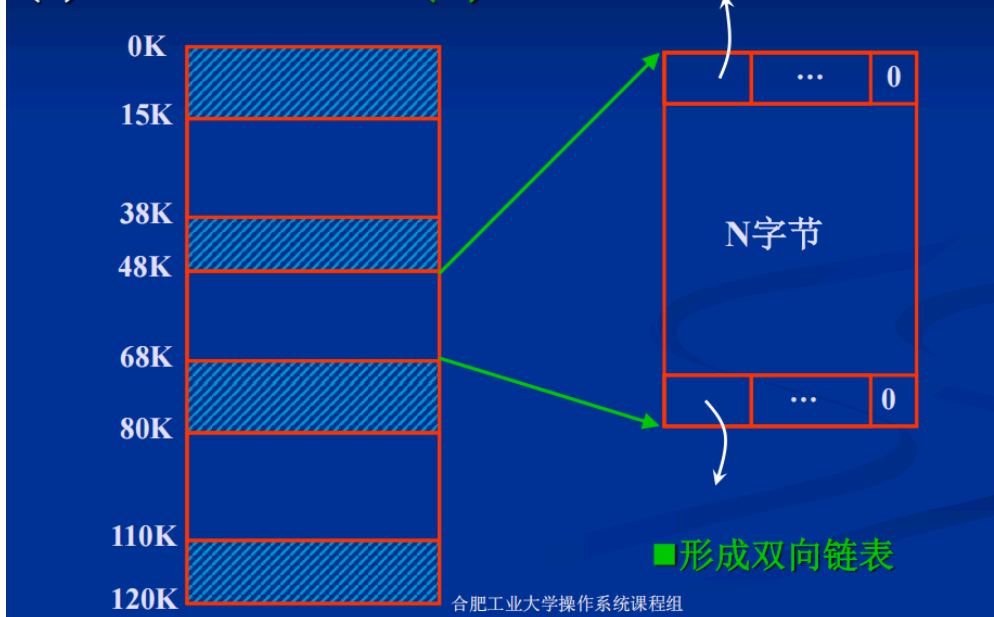
空闲分区表

始址	长度	标志
15K	23K	未分配
48K	20K	未分配
80K	30K	未分配
		空
		空

合肥工业大学操作系统课程组

- **空闲分区链**：双向链表，链接上下分区

(1) 数据结构: (b)空闲分区链



2. 算法

分区分配算法有4种

(a) 分区分配算法——首次适应算法

- **方法:**
 - 按地址递增顺序排列空闲分区链;
 - 分配内存时, 总是从低地址端开始扫描空闲区;
 - 找到的第一个大小合适的分区, 就分割分配; 否则失败
- **特点**
 - 高址端有大空闲区概率大;
 - 低址端迅速被划分, 碎片出现速度快;

(b) 分区分配算法——循环首次适应算法

- **方法:**
 - 按地址递增顺序排列空闲分区链;
 - 设置当前指针;
 - 分配内存时, 从指针所指位置开始扫描空闲区; (不是从低地址端重新开始哦)
 - 找到的第一个大小合适的分区, 就分割分配; 否则失败
- **特点:**
 - 碎片分布均匀
 - 高址端有大空闲区概率小
- **本质:** CLOCK算法

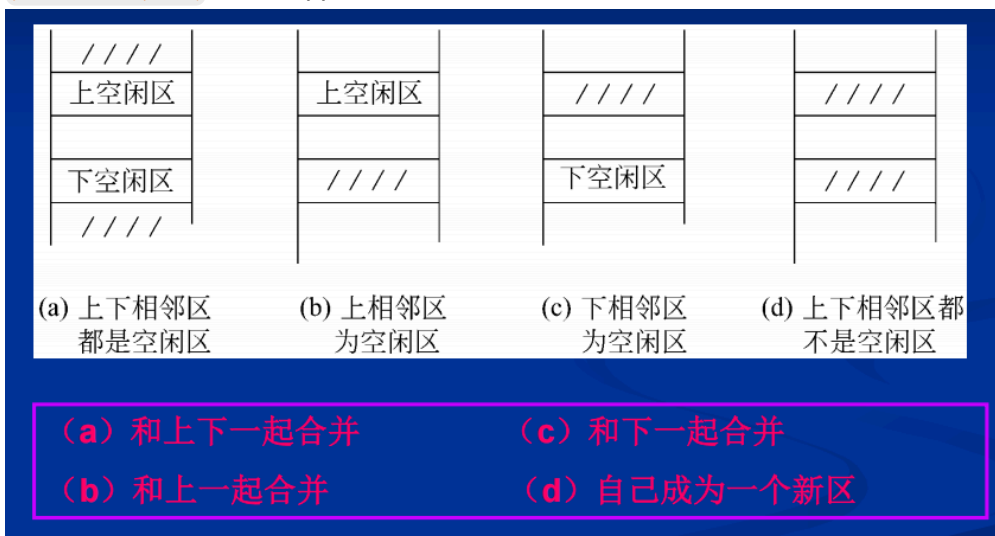
(c) 分区分配算法——最佳适应算法

- **方法：**
 - 按大小递增顺序排列空闲分区链；（这次是大小！小→大）
 - 分配内存时，从链首开始扫描空闲区；
 - 找到的第一个大小合适的分区，就分割分配；否则失败。
- **特点：**
 - 碎片迅速出现
 - “最佳”匹配

(d) 分区分配算法——最坏适应算法

- **方法：**
 - 按大小递减顺序排列空闲分区链；（大→小）
 - 分配内存时，总是分配链首空闲区；否则失败
- **特点：**
 - 碎片出现最慢
 - “最坏”匹配

分区回收算法也有四种



(4) 特点

- **“碎片”问题**
 - 经过一段时间的分配回收后，内存中存在很多很小的空闲块。
 - 它们每一个都很小，不足以满足分配要求；但其总和满足分配要求。
 - 这些空闲块被称为**碎片**，造成存储资源的浪费

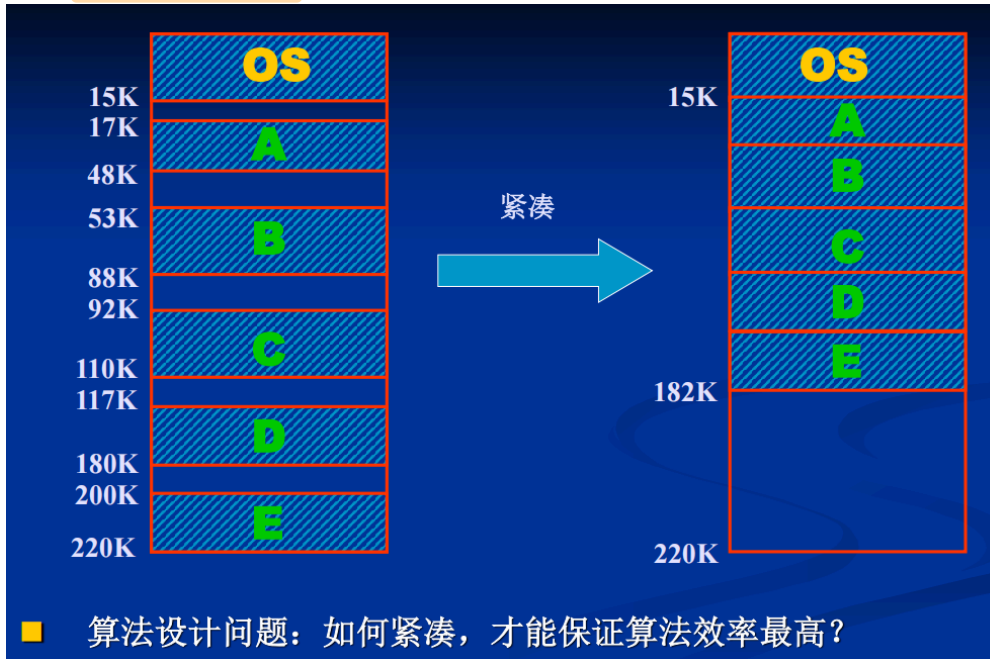
4.2.4 可重定位分区分配存储管理方案

(1) 应用背景

- 多道程序设计系统

(2) 基本思想

- 采用动态分区分配方式
- 引入“紧凑”机制，合并小的碎片空闲区
- 使用动态地址重定位



4.3 基本分页存储管理方式

4.3.0 分区式分配存在的问题

- 碎片（内外“零头”）问题难以解决

4.3.1 分页存储的基本设计思想

(1) 引入页面与页框

页/页面：原逻辑地址空间，被划分为大小相等的各部分

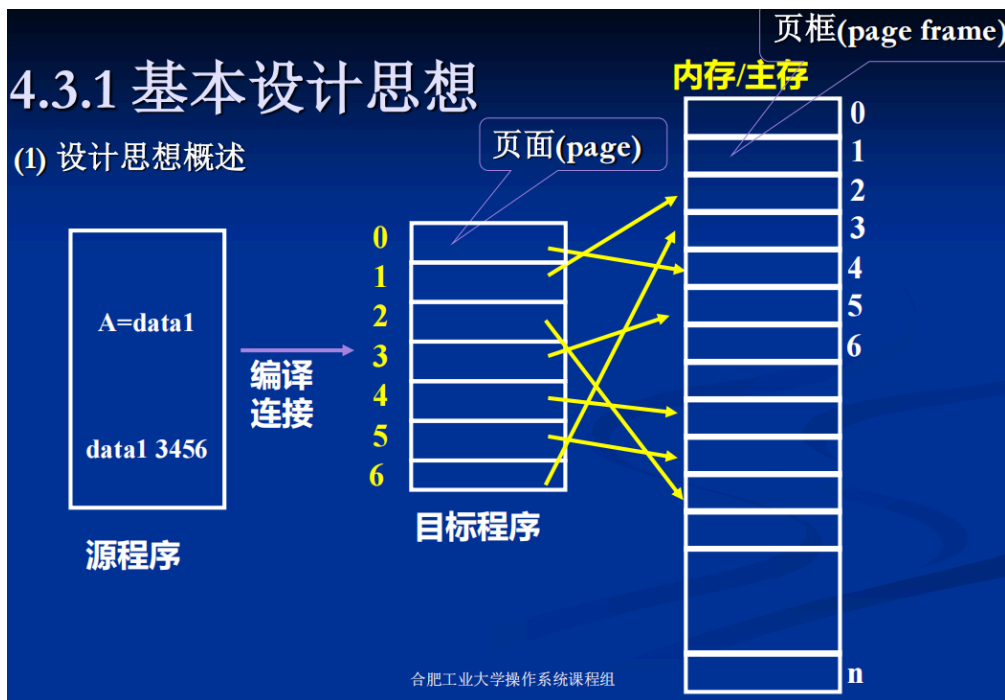
页框：原物理地址空间，被划分为大小相等的各部分，也可以成为物理块

二者关系：页面与页框一一对应

进程使用：进程申请页面，后映射到页框

4.3.1 基本设计思想

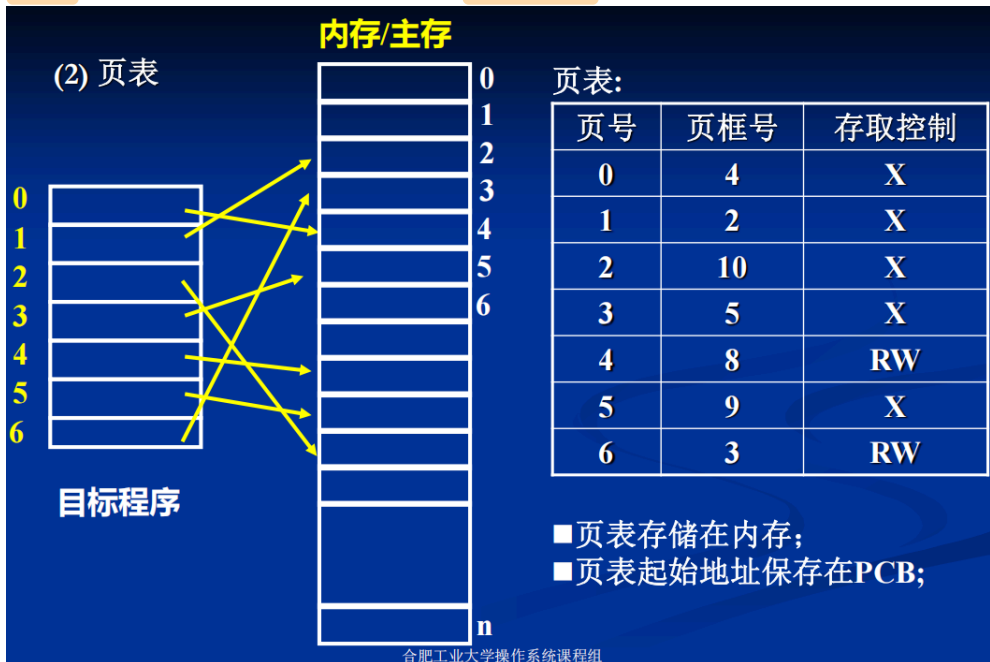
(1) 设计思想概述



(2) 引入页表

页表: 记录页面对应的页框号（物理块号），可实现页号到页框号（物理块）号的映射

- 页表存放在内存中，但是页表的起始位置保存在PCB中！



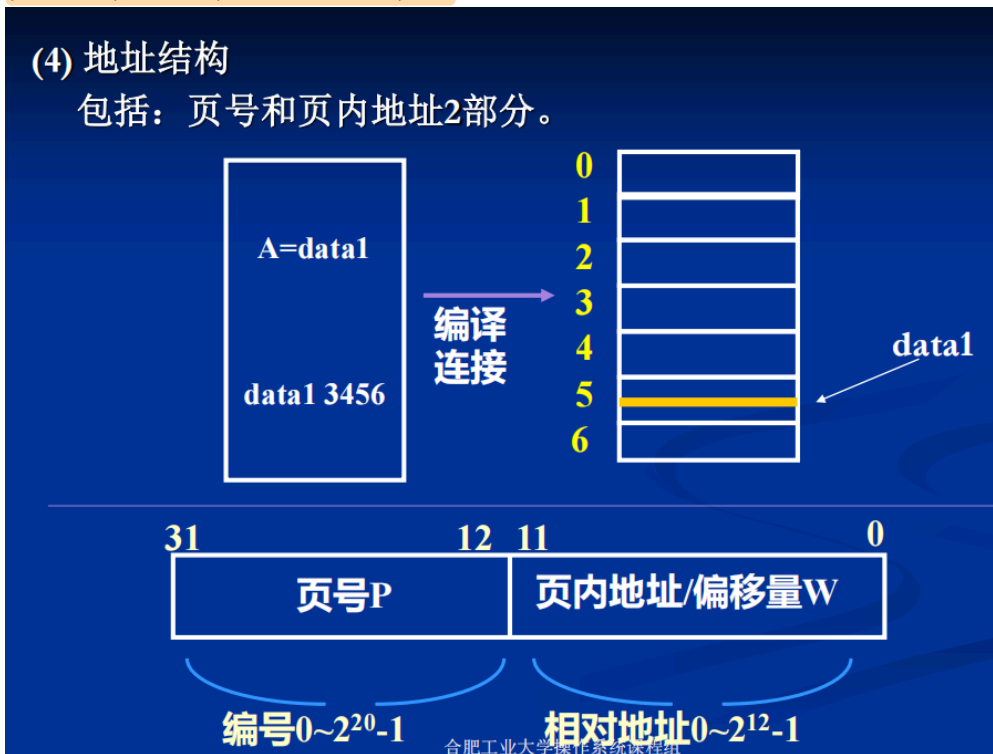
- 页表的作用 (摘自4.3-4.5 p13):** 分页系统的核心数据结构
 - 记录程序各页面所在的页框位置；
 - 支持进行地址重定位；
 - 实现页面访问控制；
 - 存储保护:** 限制程序在操作系统指定的内存区域内运行

(3) 页面尺寸

- **页面大小的影响**
 - **太小**：内存利用率（内零头）高，页面数量增加，占用内存；
 - **太大**：内存利用率（内零头）低，页面数量减少，节省内存；
- **正常尺寸**：2的整数次方，512-8K

(4) 地址结构

页面 = 页号 + 页内地址/偏移量



4.3.2 地址重定位

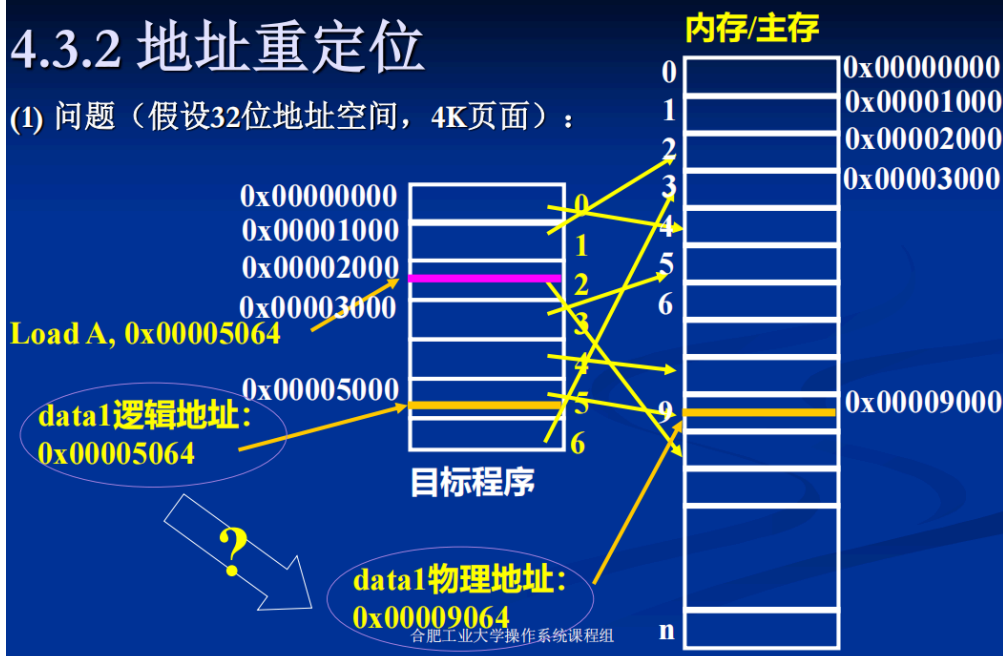
(1) 问题

如图

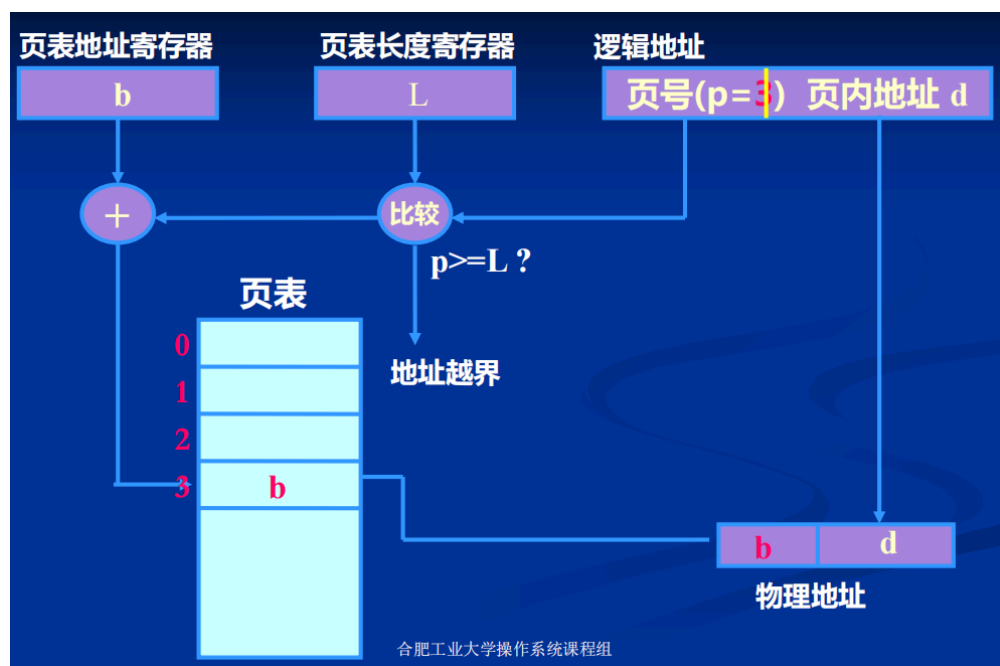
- 指令Load A, 0x00005064逻辑地址在0x00002000和0x30000000之间
- 数据data1逻辑地址为0x00005064，物理地址为0x00009000
- 逻辑地址与物理地址之间的地址重定位/地址映射如何实现呢？

4.3.2 地址重定位

(1) 问题（假设32位地址空间，4K页面）：



(2) 基本地址变换机构/地址映射机制



转换流程：

1. 逻辑地址中，**页号**输入变换机构，页内地址即**物理地址的页内地址**
2. 将页号输入**页表长度寄存器**，页号p与页表长度寄存器中存放的长度L做比较，**如果p >= L，则地址越界，如果不是则执行下一步。**
3. 使用页号在**页表地址寄存器**中，得到物理地址的**页框号**
4. **物理地址 = 页框号 + 物理地址的页内地址**



转换示例：设data1的逻辑地址为addr

- **页号**—— $\text{addr}[31:12] = 0 \times 00005$
- **页内地址/偏移量**—— $\text{addr}[11:0] = 0 \times 064$
- 查表得，页号5的页框号为9
- **物理地址**—— 0×00009064

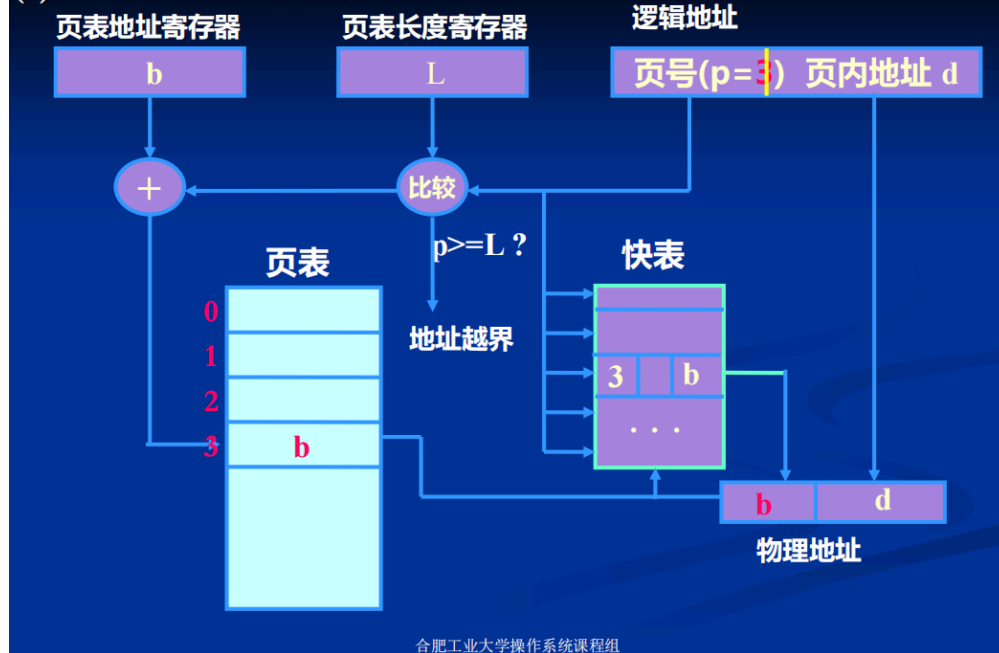
(3) 具有快表的地址变换机构

1. 快表的概念

- **别名**：
 - 联想存储器（相联存储器）(associative memory)
 - TLB(Translation lookaside buffers) (不是线程控制块!)
- **位置**：设置在CPU内部
- **功能**：具有并行查找能力，暂存当前正在使用的页表项
- **尺寸**：16-512，可达到90%以上命中率

2. 地址变换机构

(3) 具有快表的地址变换机构



可以先在快表中查找，快表命中则直接转换，加快效率

4.3.3 两级和多级页表

(1) 页表的存储问题

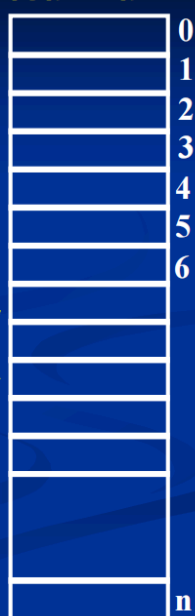
4.3.3 两级和多级页表

(1) 页表的存储问题（假设4K页面,4字节页表项）：

页表：

页号	页框号	存取控制
0	4	X
1	2	X
2	10	X
...	...	...
1023	9879	RW
...	...	...
1760	3	RW

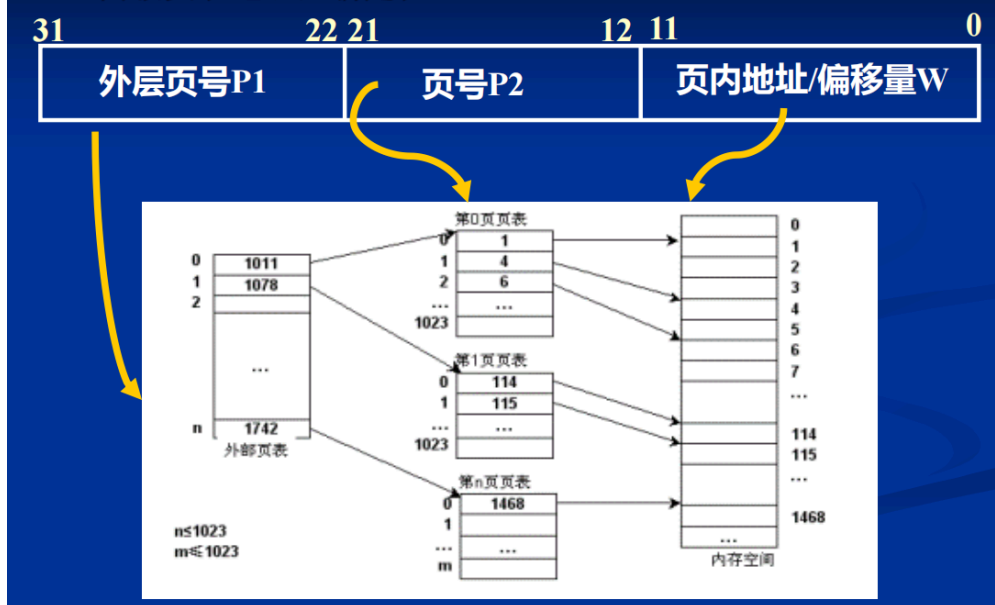
内存/主存



- 现代操作系统支持非常大的逻辑地址空间，导致页表非常大
- 页表很大时需要非常大的连续空间
- 一段时间内，并非所有页表都用得到，可以只调入一部分页表进入内存

(2) 两级页表

■ 两级页表地址重新定位



1. **页表分页**：将页表进行分页，使每个页面与页框（内存物理块）大小相同，并将他们离散地分配在不同的物理块中
2. **建立外部页表**：外部页表的页表项记录页表各页面的物理块号

(3) 多级页表

可基于二级页表，建立多级页表

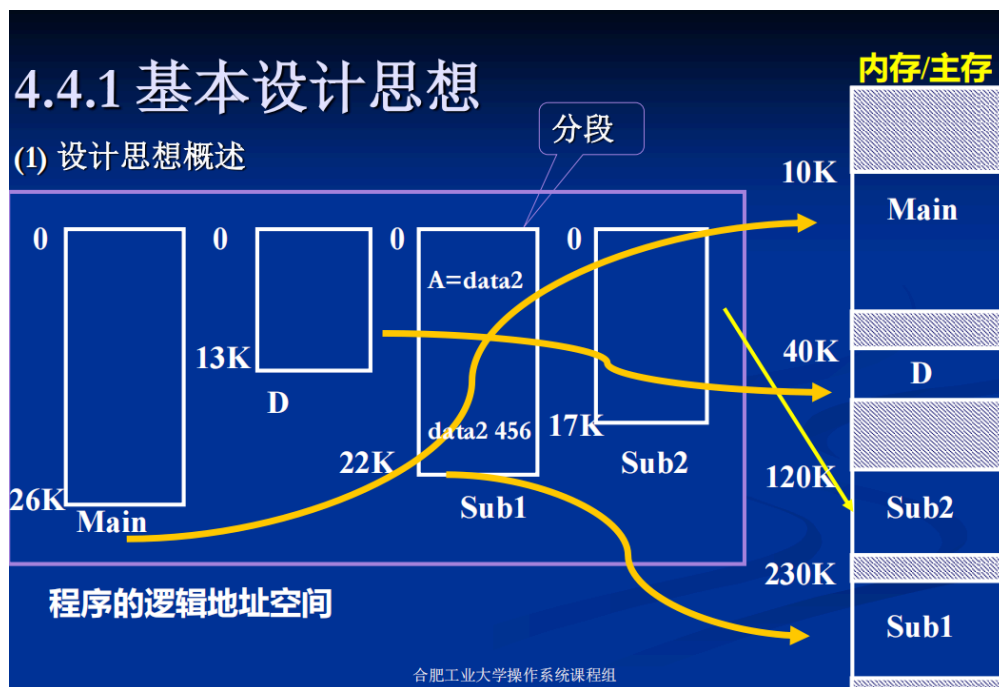
4.4 基本分段存储管理方式

4.4.0 分页系统的不足

1. 信息共享和保护困难
2. 编程不便

4.4.1 设计思想

(1) 设计思想概述



将程序 **分段** 存储在内存的不同位置

(2) 分段 (Segment)

- **定义**：分段是一段 **有意义的信息集合**；
- **划分**：分段的划分 **由程序员完成**；
- **长度**：分段的 **长度不定**；
- **指令**：指令 **不存在跨分段情况**；

(3) 段表

(3) 段表

段表:

段号	段基址	段长	存取控制
0	10K	26K	X
1	40K	13K	RW
2	120K	17K	X
3	230K	22K	X

- 段表存储在内存;
- 段表起始地址保存在PCB;

内存/主存

10K

0 Main

40K

1 D

120K

2 Sub2

230K

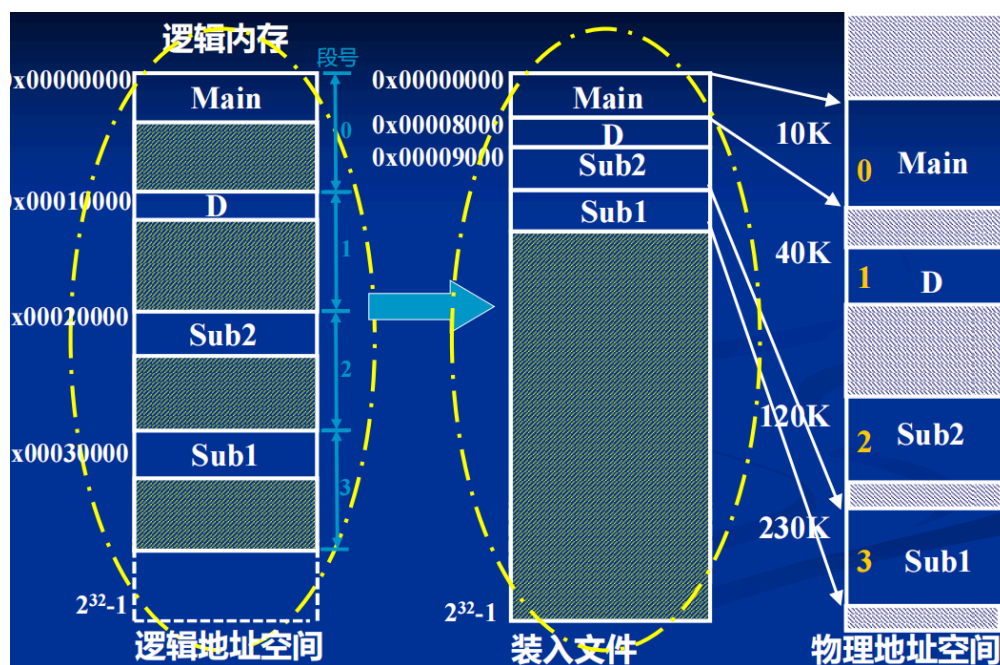
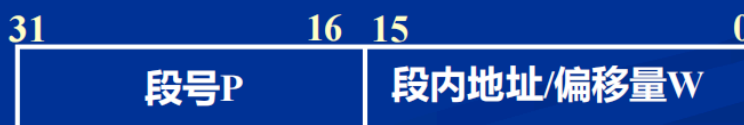
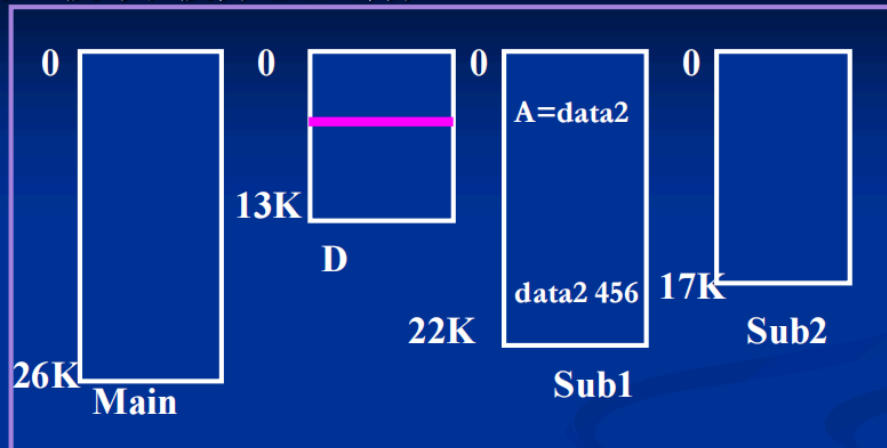
3 Sub1

合肥工业大学操作系统课程组

- 段=段基址+段长
- 存储位置: 段表在内存中, 段表起始位置在PCB中
- 作用: 分段系统的核心数据结构
 - 记录程序各分段所在的内存位置;
 - 支持进行地址重定位;
 - 实现分段访问控制;
 - 存储保护: 限制程序在操作系统指定的内存区域内运行

(4) 地址结构

包括：段号和段内地址2部分。

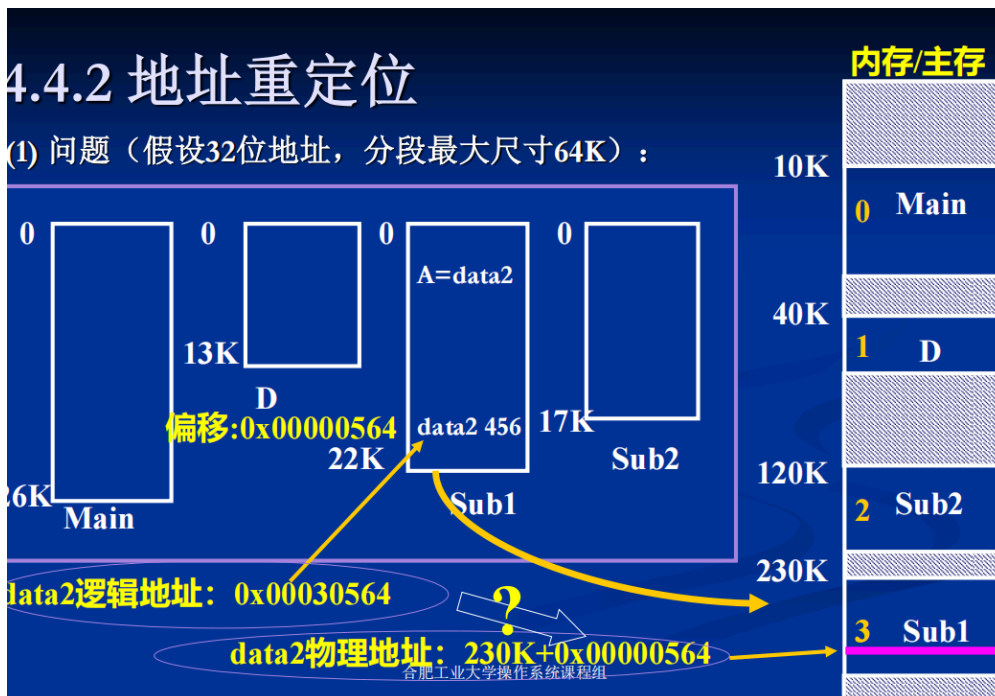


地址结构：假设地址为addr

- 段号：addr[31:16]
- 段内地址：addr[15:0]

4.4.2 地址重定位

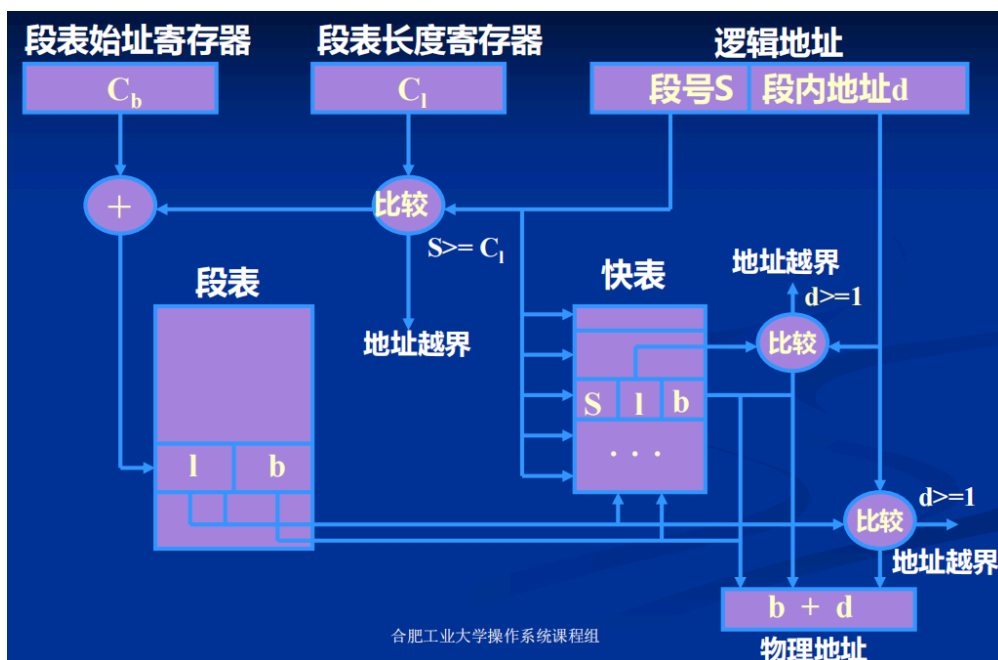
(1) 问题



对于段3（Sub1）段中的某个数据data2

- 逻辑地址为0x00030564
- 物理地址为230K+0x00000564
- 230K是根据 $addr[31:16]=0 \times 0003$ 的物理块号得到的
如何实现地址重定位的过程？

(2) 分段地址变换及存储保护机构



(3) 地址变换机构/地址映射机制：计算实例



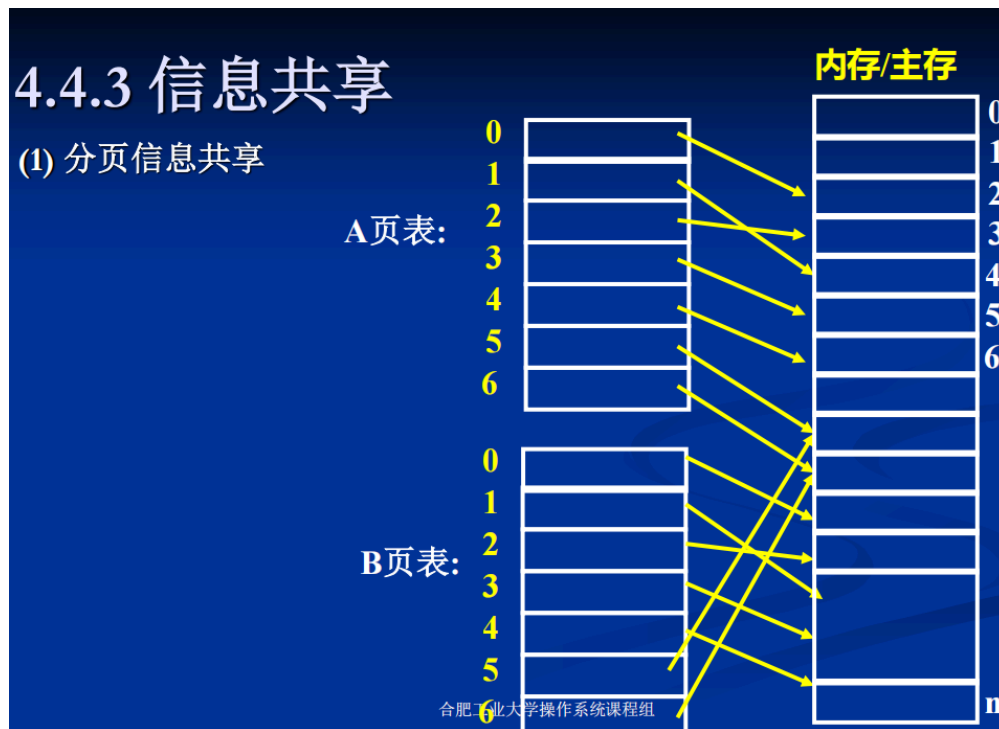
- 逻辑地址 0x00030564
 - 0x0003对应段基址230K
 - 0x0564不超过22K，为有效的相对地址
- 物理地址 为230K+0x00000564

(4) 分段和分页区别

项目	分段	分页
信息单位	信息的逻辑单位	信息的物理单位
大小	不定	固定
可见性	程序员确定，可见	系统确定，程序员不可见
地址空间	二维地址空间	一维线性地址空间
信息共享保护	方便	不方便

4.4.3 信息共享

(1) 分页信息共享



解释: A和B两个页表共享内存中部分物理块的信息

(2) 分段信息共享

4.4.3 信息共享

(2) 分段信息共享

A段表:

段号	段基址	段长	存取控制
0	10K	26K	X
1	40K	13K	RW
2	120K	17K	X
3	230K	22K	X

B段表:

段号	段基址	段长	存取控制
0	2100K	7K	X
1	1040K	18K	RW
2	120K	17K	X
3	230K	22K	X

内存/主存



解释: 两个段表使用的部分段相同，信息可共享

4.5 段页式存储管理方式

4.5.0 分段系统存在的问题

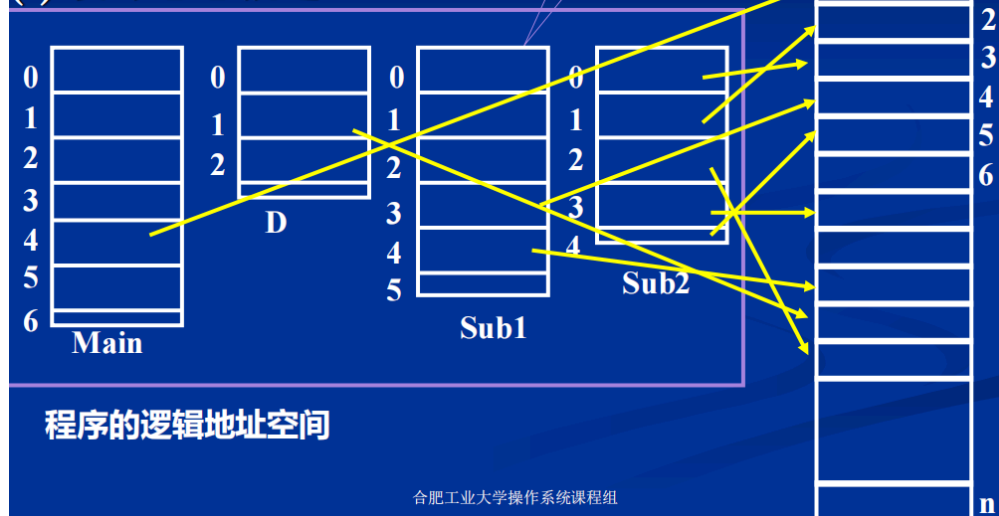
- **问题**
 - 碎片（内外“零头”）问题难以解决

4.5.1 设计思想

(1) 设计思想概述

4.5.1 基本设计思想

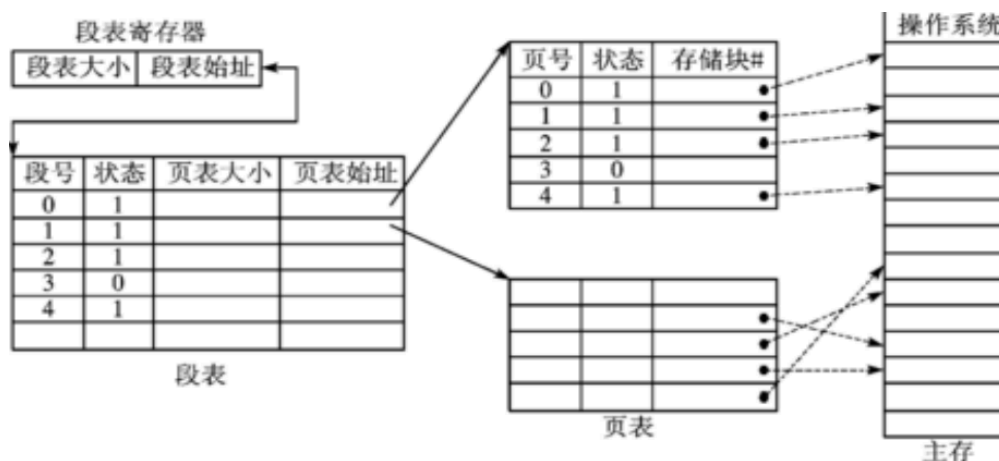
(1) 设计思想概述



实现方式：

- 逻辑地址空间分成若干个段
- 各段分成多个页

(2) 段表和页表



- **多个阶段：**
 1. 从段表中找到**外部页表**
 2. 外部页表对应**不同的页**
 3. 不同页的页表项映射**不同物理块**

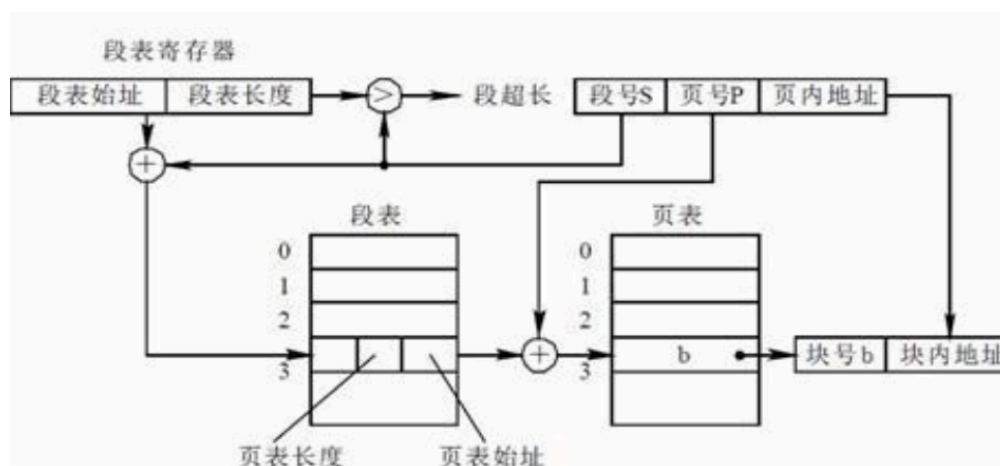
(3) 地址结构



地址结构：

- 段号+页号：addr[31:16]
- 页内地址/偏移量：addr[15:0]

4.5.2 地址变换



4.6 虚拟存储器的基本概念 (重重重点, 小时候抱过出题老头)

4.6.0 存储管理的两个基本问题

1. 大作业如何在小主存上运行?
2. 如何在给定大小主存上运行更多程序?

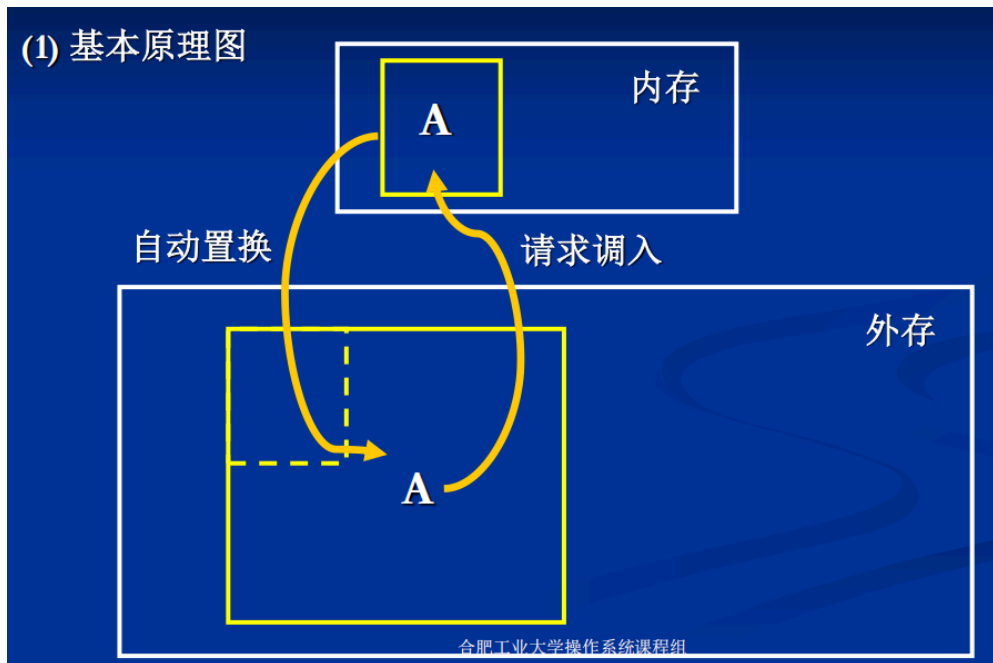
4.6.1 虚拟存储器的理论基础——局部性原理

- 回忆体系结构
- 程序运行的基本特征：局部性原理
- 定义：在一段时间内一个程序的执行往往呈现出高度的局部性，表现在时间与空间两方面
 - 时间局部性：
 - 一条指令被执行了，则在不久的将来它可能再被执行；
 - 在一段时间内，访问的代码范围是有限的。
 - 空间局部性：

- 若某一存储单元被使用，在一定时间内，与该存储单元相邻的单元可能被使用
- **启示**：是否可不将程序所有代码同时装入主存（因为只有有限的代码和相邻的单元会被使用）

4.6.2 虚拟存储器概念

(1) 基本原理



解释：

- 虚拟存储器就是图中的**外存**
- 可以和内存进行**自动置换和请求调入**

(2) 虚拟存储器的定义

- **虚拟存储器**，是指具有**请求调入功能**和**置换功能**，**能从逻辑上对内存容量加以扩充**的一种存储器系统。
- 其**逻辑容量**由**内存容量和外存容量之和**所决定
 - 其运行速度接近于内存速度
 - 而每位的成本却又接近于外存。
 - **虚拟存储技术**是一种**性能非常优越的存储器管理技术**，故被广泛地应用于大、中、小型机器和微型机中。

(3) 虚拟存储器的特征

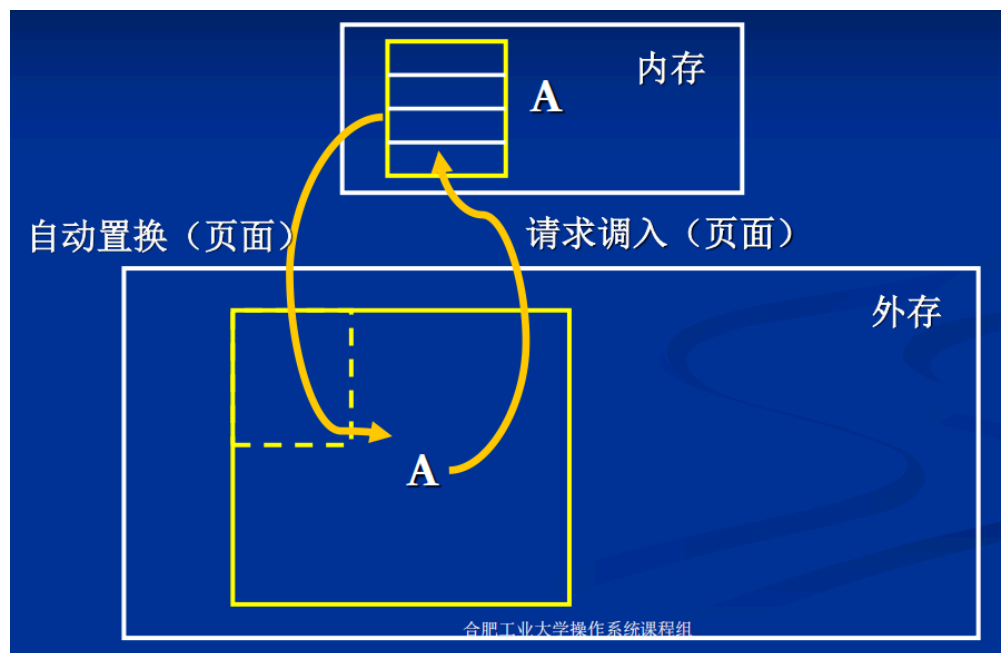
1. **多次性**
2. **对换性**
3. **虚拟性**

(4) 虚拟存储器的实现

对应4.5节中的几种实现方法

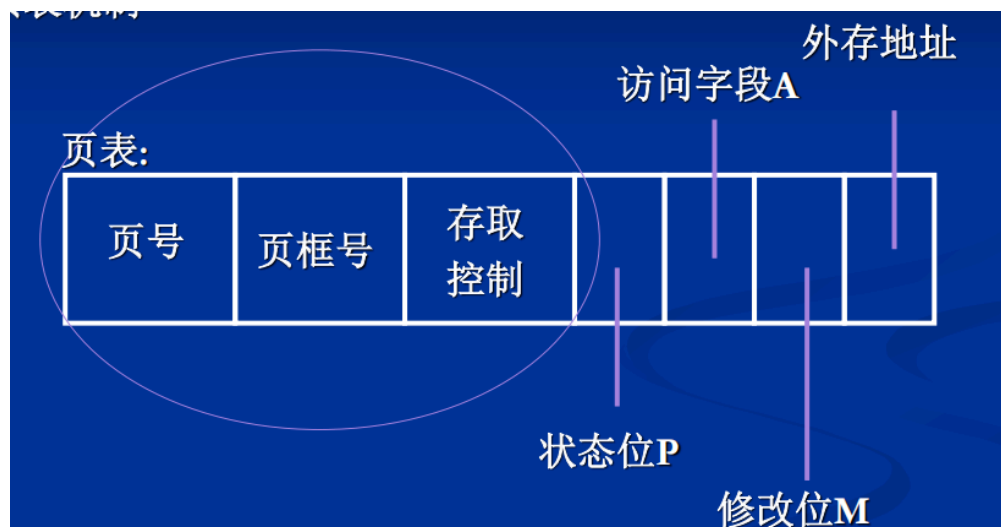
1. **分页请求系统（页式虚拟存储器，请求分页系统）**：
 - 在基本分页系统基础上，增加以页面为单位的请求调入和自动置换功能。
2. **请求分段系统（段式虚拟存储器，分段请求系统）**：
 - 在基本分段系统基础上，增加以分段为单位的请求调入和自动置换功能。
3. **段页式虚拟存储器**：
 - 在段页式系统基础上，增加以分页为单位的请求调入和自动置换功能。

4.7 请求分页存储管理方式



4.7.1 请求分页中的硬件支持

(1) 页表机制



新增字段

- **状态位P**：判断当前页是否已调入内存
- **访问字段A**：记录本页在一段时间内被访问的次数or最近已多久未被访问
- **修改位M**：标记该页调入内存后是否修改过
- **外存地址**：记录该页在外存的外存地址

(2) 缺页中断机制

缺页中断产生的原因：要访问的页面不在内存里

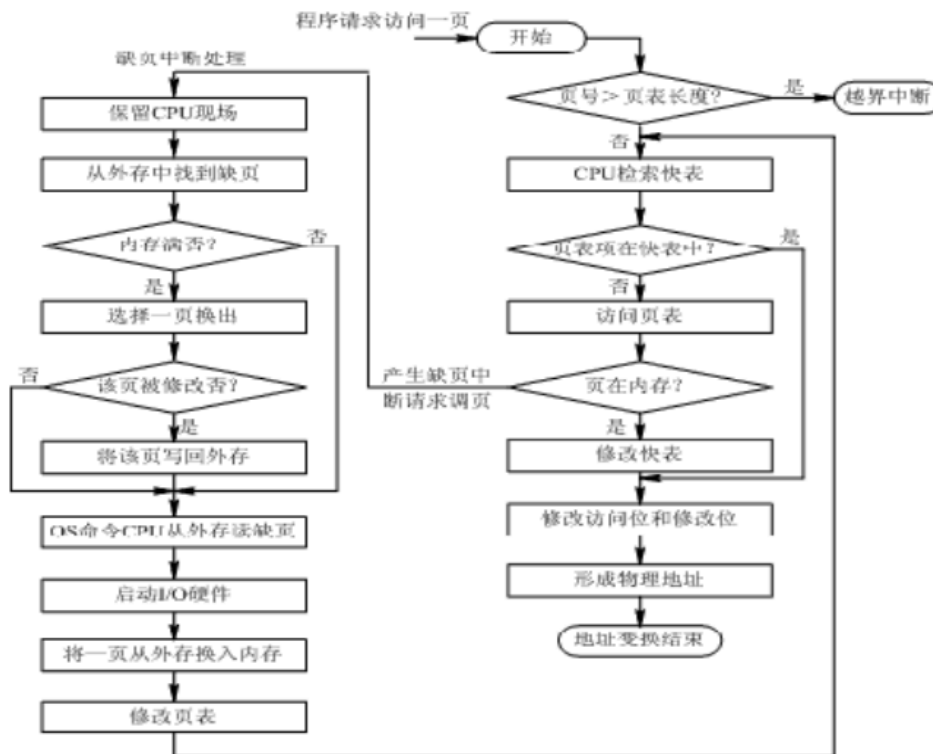
缺页中断过程：

1. 缺页进程阻塞
2. 内存中**有空闲页框**，分配一个，**并从外存调入缺少的页面**，并修改页表中的页表项
3. 内存中**无空闲页框**，运用**页面置换算法**淘汰
4. 该页如果在内存中修改过，则还需要写回外存

缺页中断与普通中断的区别：

5. 在指令执行期间而非执行结束产生：发出中断并响应和处理中断，返回
6. 一条指令可能发出**多次缺页中断**

(3) 地址重定位



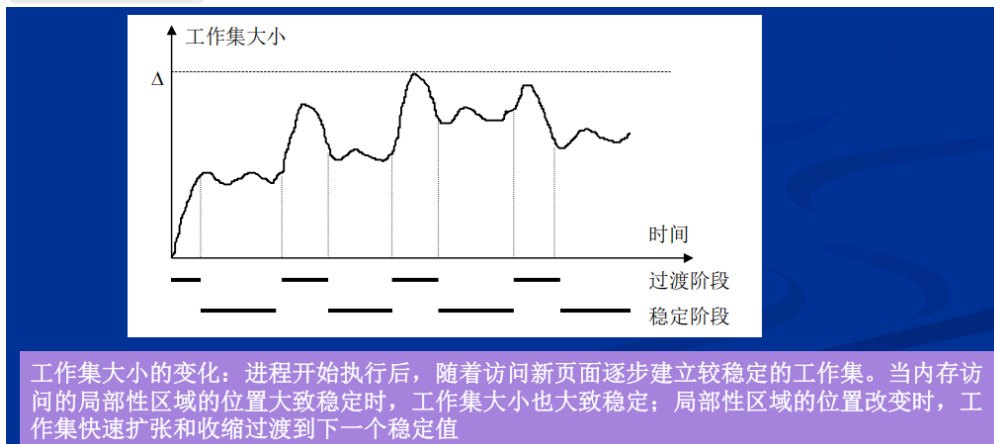
4.7.2 内存分配策略和算法

(1) 最小物理块个数

- **定义**：保证程序正常运行需要的最小物理块个数
- **影响因素**：由机器指令的结构决定
 - 单字节指令，直接寻址：2块；
 - 单字节指令，间接寻址：3块；
 - 多字节指令，直接寻址：3块；
 - 多字节指令，间接寻址：4块；

(2) 工作集

- **定义**：进程执行时需要的物理块的数量
- **进程的工作集**：驻留在物理内存中的虚拟页面的子集



(3) 物理块分配策略

1. 固定分配局部置换

策略：

- 为进程分配物理块；
- 在进程运行期间，物理块个数不变；
- 新页面，只能置换入该进程已经分配的物理块

特点：物理块个数难以指定

2. 可变分配全局置换

策略：

- 为进程分配物理块；
- 在进程运行期间，物理块个数随时变化；

- 新页面，在**全局范围内**置换物理块；

特点：

- 进程之间的执行互相影响；
- **页面“抖动”**

3. 可变分配局部置换

策略：

- 为进程分配物理块；
- 在进程运行期间，**物理块个数可变化**；
- 新页面，在**该进程已经分配物理块内**置换；
- 引入**缺页率**，保证：**低限阈值 $\leq$ 缺页率 $\leq$ 高限阈值**

特点：

- 避免物理块**初始设置不合理**的难题

(4) 物理块分配算法

- 平均分配
- 按比例分配
- 加权分配

4.7.3 页面策略

(1) 何时调入

- **预先调页**
 - **一次调入多个页面**；
 - 减少后期I/O；
- **请求式调页**
 - **需要时，调入**；

(2) 何处调入

- 外存对换区
- 文件区
- UNIX方式：第一次在文件区，后面在对换区

4.7.4 页面置换算法

(1) 最佳置换算法

- **基本思想**
 - 选择**以后再也不用的**页面；

- 没有的话，选择以后最长时间不用的页面；
- **实现**
 - 无法实现，因为页面的访问顺序无法预知
- **特点**
 - 无法实现，仅具有理论意义；

(2) 先进先出置换算法 (FIFO)

- **基本思想**
 - 基于：程序的顺序执行特点
 - 选择到达内存最早的页面，予以淘汰；
- **实现**
 - 页面在内存中按时间排序；
- **特点**
 - 效果不佳（程序不是严格顺序执行）；

(3) 最近最久未使用置换算法

- **基本思想**
 - 基于：程序运行的局部性原理；
 - 选择最近以来最久未使用的页面，予以淘汰；
- **实现**
 - 移位寄存器，栈；
- **特点**
 - 调度性能较好；
- **移位寄存器实现方案**
 - 每个物理块设置一个移位寄存器，初值为0；
 - 访问一次页面，高位置1；
 - 定时（100ms）将所有物理块的移位寄存器右移1位，高位补0；
 - 选择移位寄存器数值最小的物理块，淘汰其中页面

■ 栈实现方案

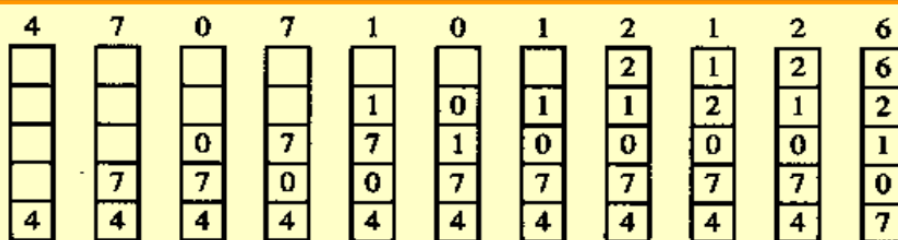
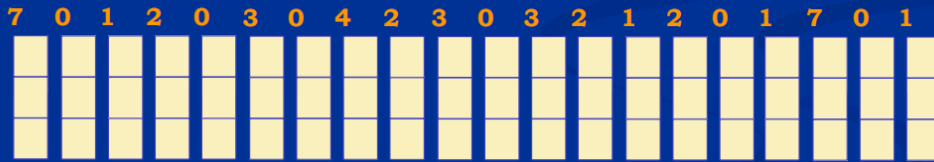


图 4 - 29 用栈保存当前使用页面时栈的变化情况

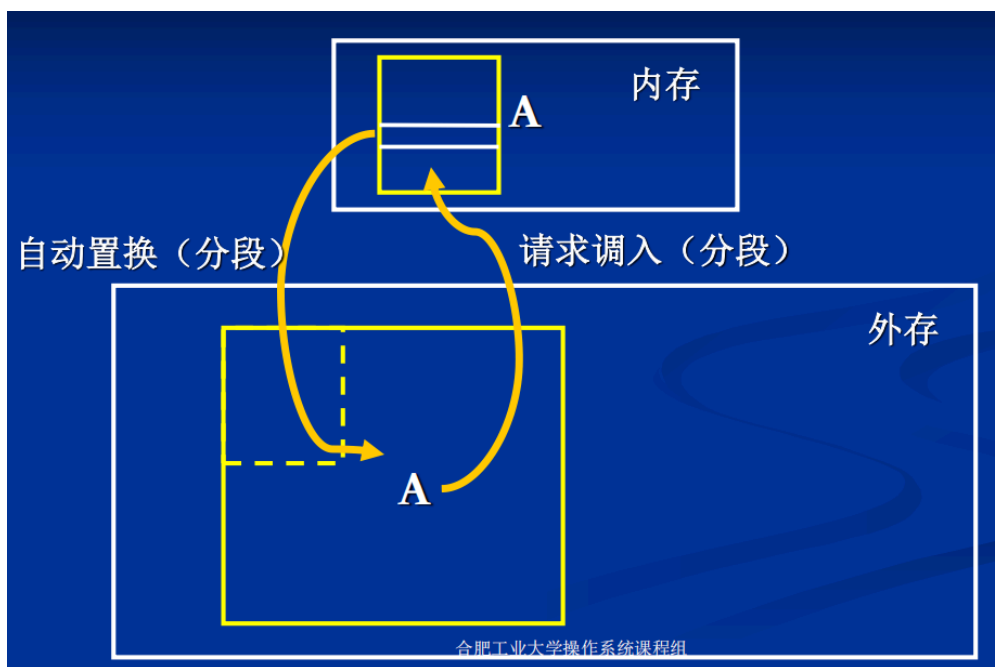
(4) 置换算法性能比较

■ 实例1:

某分页系统，进程分到3个存储块，其页面访问序列为：7-0-1-2-0-3-0-4-2-3-0-3-2-1-2-0-1-7-0-1，请分别计算三种页面调度算法OPT、FIFO、LRU下的缺页率（缺页率=缺页置换次数/页面访问总数*100%）。

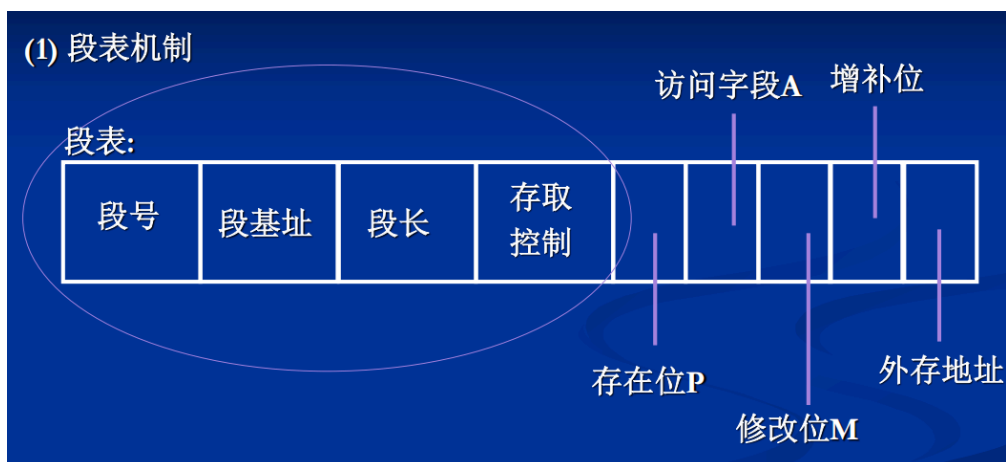


4.8 请求分段存储管理方式



4.8.1 请求分段中的硬件支持

(1) 段表机制



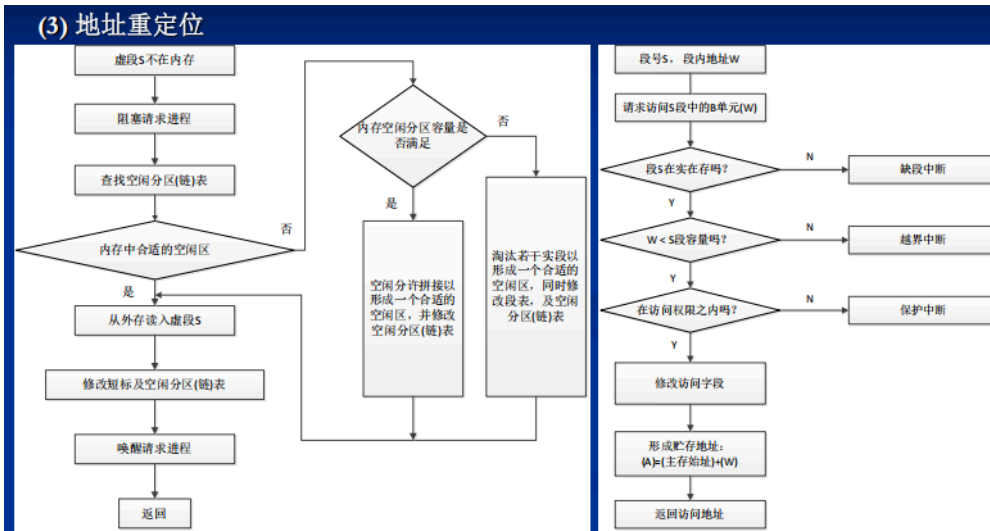
(2) 缺段中断机制

过程：类似缺页中断

缺段中断与普通中断的区别：

- **指令执行期间**：发出中断并响应和处理中断，返回
- 一条指令可能发出**多次缺段中断**

(3) 地址重定位



4.8.2 分段共享与保护

(1) 数据结构

- **共享段表 (所有进程共享)**
 - 整个系统设置1张共享段表;
 - 记录: 分段信息, 共享该分段进程的信息(存取控制, 进程数量count, 段号)。
- **进程段表 (私有段表)**
 - 与前面所述段表相同;

(2) 算法: 共享分段的分配与回收

- **分配算法**
 1. **检索共享分段**, 如果共享分段存在转3;
 2. **创建共享段表的段表项**, 填写分段信息, 分配相应内存并调入分段;
 3. 记录**本进程信息**到共享段表的段表项, count+1;
 4. 复制共享段表的段表项相应信息(段基址、段长、存取控制)到**进程段表**;
- **回收算法**
 1. 撤消进程时, 将所有该进程共享之分段的count-1;

2. 当某分段count =0时，回收该分段所占据内存；